CS4287

Assignment 1: Convolutional Neural Networks

Fire and Smoke Detection

A ResNet-50 and YOLOv8 Hybrid Model

Work Completed by

Ciaran Whelan	23370211
Emma O'Reilly	23304472
Hersh Singh	23371137

Contents

1 Exploratory Data Analysis 1
1.1 Sample Distribution 1
1.2 Bounding Boxes 3
1.2.1 Annotation Validation and Cleaning 3
1.2.2 Distribution of Bounding Boxes 3
1.2.3 Spatial Distribution of Bounding Boxes 6
1.3 Image Properties 6
1.3.1 Property Distributions 7
1.3.1.1 Brightness 7
1.3.1.2 Contrast 7
1.3.1.3 Sharpness 7
1.3.2 Properties Across Splits 8
1.3.3 Properties of Annotations 8
1.4 Correlations 9
1.4.1 Box area compared to Box Count and Image Area 9
1.4.2 Annotations compared to their backgrounds 10
2 Preprocessing 11
2.1 Resizing and Letterboxing 11
2.2 ColourJitter 11
2.3 Normalisation 12
2.4 Random Augmentations 12
3 Network Structure 12
3.1 ResNet-50 Backbone 13
3.2 Adapter 13
3.3 YOLOv8 Neck 13
3.4 YOLOv8 Head 14
3.5 Transfer Learning 14
3.5.1 Weight Initialisation 15
3.6 Residuals 15
3.6.1 The Vanishing Gradient Problem 15
3.6.2 How Residual Learning Helps 15
3.6.3 Residuals in Our Model 15
3.7 Activation Functions 16
3.7.1 ReLU (backbone) 16
3.7.2 SiLU (adapter, neck and head) 16
3.7.3 Sigmoid (head) 16
3.7.4 Softmax (head) 16
3.8 Batch Normalisation 17
3.9 Regularisation 17
3.10 Hyperparameters 17
4 Loss Function, Optimiser and Cross-Fold Validation 18
4.1 Loss Function 18
4.1.1 Alternative Loss Functions 18
4.2 Optimiser 18
4.2.1 Alternative Optimisers 18
4.3 Cross-Validation 19
4.4 Model Comparison 20

5 Results 21
5.1 Model Results 21
5.2 Results from Other Models 22
5.2.1 YOLOv8 Cross-Dataset Evaluation 23
5.2.2 TFNet: Transformer-Based Multi-Scale Feature Fusion 23
5.3 Near-Duplicates in the Test Split 24
6 Independent Experiments 24
6.1 Baseline Run 25
6.2 LR Plateau Reductions 25
6.3 Random Horizontal Flips 26
6.4 Increasing the Resolution 26
6.5 Cosine Annealing 27
6.6 Random Zoom-ins 28
6.7 Final Training Run 29
6.8 Hyperparameter Tuning 30
7 Level of Difficulty 31
8 Statement of Work 31
9 Generative AI Use 33
Bibliography 36
Code Sources 36

1 Exploratory Data Analysis

This analysis looks at the Smoke-Fire Detection Dataset[1], which is designed for training and evaluating YOLO-based object detection models. The dataset contains over 20,000 images showing a variety of environments and scenarios, including scenes containing, smoke, fire, both or neither. Some images represent a sequence of observations of the same scene, while others capture identical locations at different points in time. The dataset combines images from multiple publically available datasets into a single collection, giving a diverse range of scenarios for fire and smoke detection.

The analysis follows as:

1. Examine the distribution of samples
2. Bounding-box properties for fire/smoke
3. Image level statistics (brightness, contrast, etc.)
4. Correlation between features of the dataset

1.1 Sample Distribution

This section examines the overall composition of the dataset in terms of its train/val/test split and the proportion of samples within each split and the number of bounding boxes for each category, fire (1) and smoke (0). The distributions help see whether there are any class biases within the dataset and if any stratification or changes are needed to balance out the subsets. This is important for identifying potential class imbalances that could bias the model towards more frequently represented classes, reducing its ability to detect underrepresented categories.

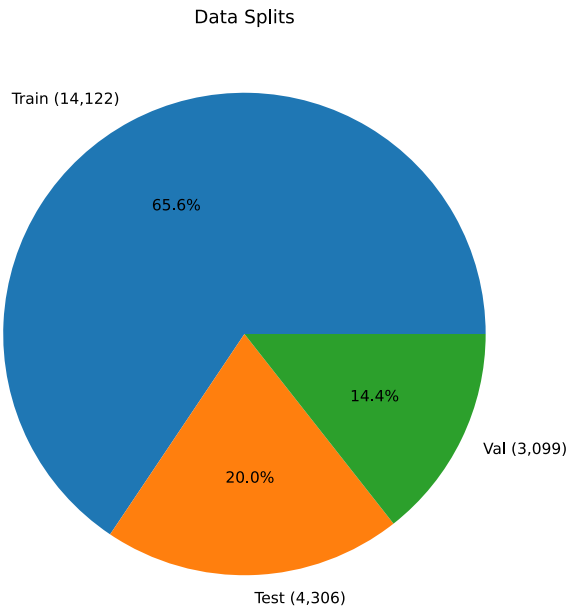


Figure 1: Distribution of images across the training, validation, and test splits.

As seen in Figure 1 the dataset from Kaggle is already pre-split into predefined training, validation and test subsets. However, for future cross-fold validation, train and validation subsets combined contain ~17,000 images with validation making up ~18% of that, so a five-fold cross-validation can help keep that split in the data.

Next is examining how the images are annotated within these splits Figure 2. There is overall an even-distribution of images containing fire and/or smoke (54.3%) versus containing neither (45.7%). This suggests that no additional balancing is needed between these two categories.

However, the second pie-chart which observes only the images containing fire and/or smoke, initially suggests a significant class imbalance, with smoke-only images accounting for 50.2% of the images compared to just 10% containing just fire. When considering both class, roughly 90% of the images contain at least one smoke-annotation. Further analysis of the bounding-box annotation is needed to determine whether this imbalance is also seen in the number of annotations of each class.

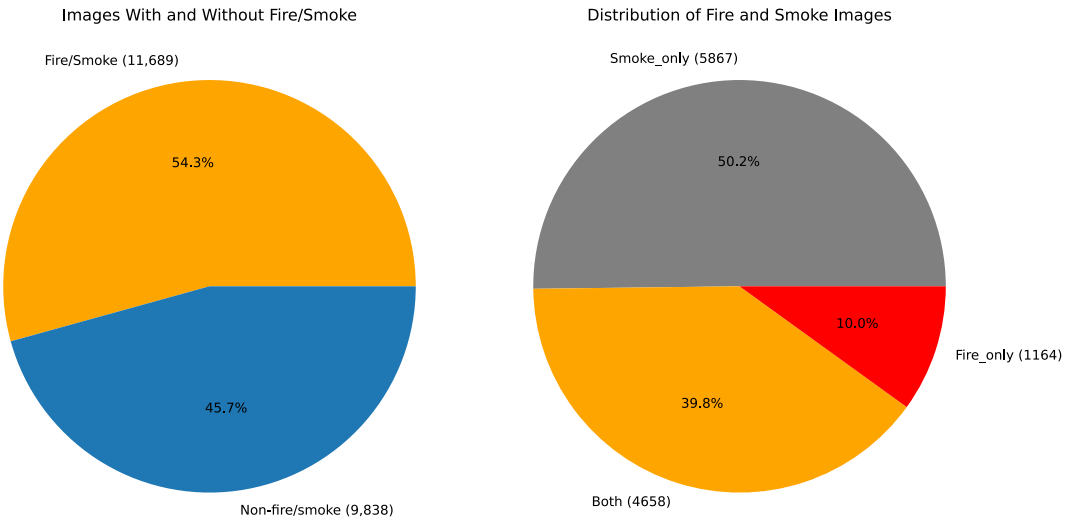


Figure 2: Distribution of fire and smoke images:
(a) Proportion of images containing fire or smoke versus neither.
(b) Distribution of smoke-only, fire-only and combined images

The next stage now looks at how these annotations aredistributed across the training, validation and test splits. This is to ensure that each split gets an even sample of images such that there are no biases in that data at this level. For example, if the training set contained mostly smoke-only images while the test set has mostly fire-only, the model could struggle to detect fire due to insufficient exposure to representative examples during training.

However, in the actual data shown in Figure 3 this is not a problem as each split maintains across all three splits, which is distributed to; Both(~20%), Smoke-only(~30%), Fire-only(~5%), None(~45%). This suggests that the predefined splits are well-balanced relative to the overall dataset composition, but will need to be kept in mind for cross-fold validation to maintain these compositions.

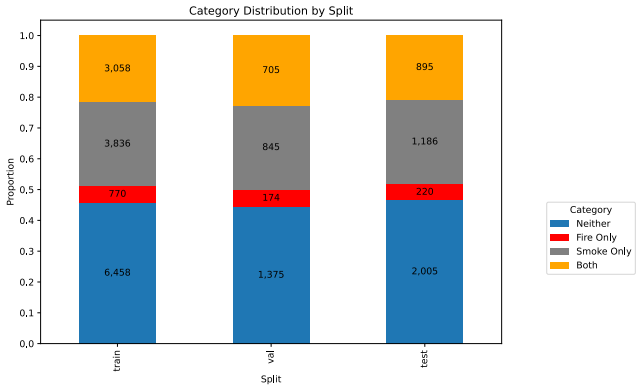


Figure 3: Normalised proportions of image-annotations across splits

As mentioned in the discussion around Figure 2 there could be a problem with the distribution of fire and smoke annotations. However, Figure 4 reveals the opposite of this trend is true where there tends to be more fire annotations (55.3%) compared to smoke (44.7%). This indicates that although smoke appears in more images, fire is annotated more frequently overall.

This is not a huge difference and there is no reason to try and enforce this to an exact 50-50 split. Such changes could cause a loss of valuable samples and reduce the variety of contexts represented within the dataset.

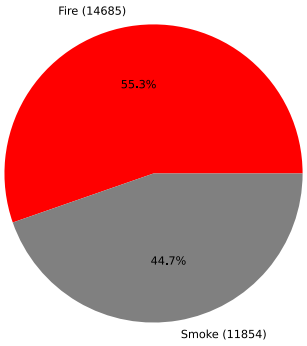


Figure 4: Fire vs Smoke box annotations

1.2 Bounding Boxes

This section examines the distribution and validity of bounding-box annotations throughout the dataset, including their spatial locations and the proportion of each image occupied by an average bounding box. An important part of this analysis is identifying invalid annotations, such as bounding boxes with zero area or coordinates extending beyond the image boundaries. These errors must be addressed to ensure that the annotations provide meaningful information during model training.

1.2.1 Annotation Validation and Cleaning

The analysis identified 18 bounding boxes with zero area. Examples of these annotations are shown in Figure 5. Since these boxes contain no spatial information about the annotated objects and appear to result from annotation errors, they were removed from the dataset to prevent potential issues during model training.

Additionally, 379 bounding boxes across 322 images were found to extend beyond the image boundaries. These annotations were corrected by clipping their coordinates to the valid image boundaries and recalculating their centres and dimensions. This ensures that the bounding boxes represent only regions within the image, preventing the model from learning invalid spatial coordinates.



Figure 5: Examples of images with invalid bounding-boxes (area=0)

1.2.2 Distribution of Bounding Boxes

The distribution of bounding-box areas is heavily right-skewed, as shown in Figure 6, with a high probability density for bounding boxes occupying a small proportion of their respective images. This

density decreases rapidly as bounding-box area increases, although a long tail extends towards annotations occupying nearly the entire image. The probability density represents the concentration of bounding-box areas, where the area under the curve over a given interval represents the probability of an annotation falling within that range.

This is an important consideration during preprocessing, as reducing image resolution could result in already small annotated regions containing very little visual information, potentially making them more difficult for the model to detect.

Further analysis by category in Figure 7 reveals substantial differences between fire and smoke annotations. Fire bounding boxes are heavily concentrated towards the lower end of the distribution, with a sharp peak near zero and a rapid decrease in density as the bounding-box area increases. In contrast, smoke bounding boxes have a much broader distribution, with a lower peak and a gradual decrease in density across a wider range of areas. Although both categories contain some exceptionally large bounding boxes, the majority of fire annotations occupy substantially smaller proportions of their images compared with smoke annotations.

These findings suggest that bounding-box size may influence detection performance, particularly for smaller fire annotations. During model evaluation, detection performance should therefore be examined across different bounding-box sizes to determine whether smaller objects are disproportionately difficult to detect and whether image preprocessing contributes to this limitation.

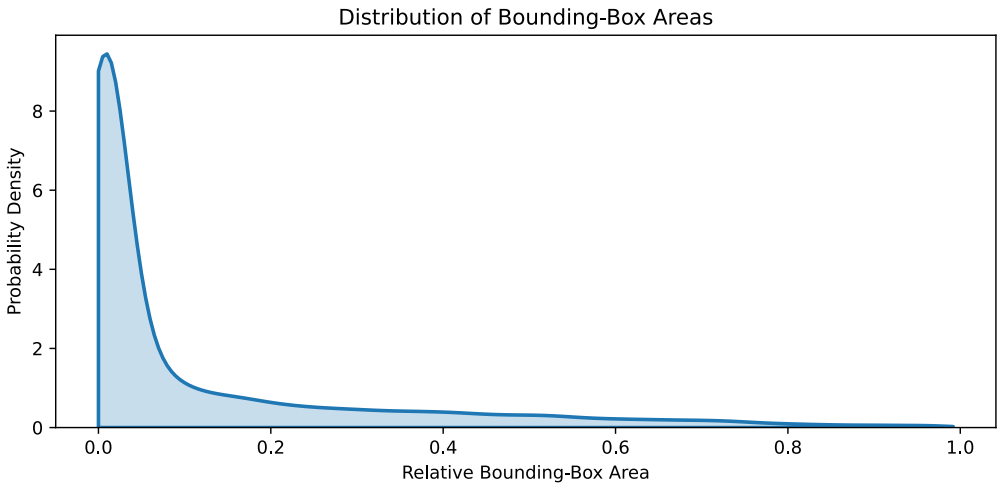


Figure 6: Kernel density estimation of relative bounding-box areas across the dataset.

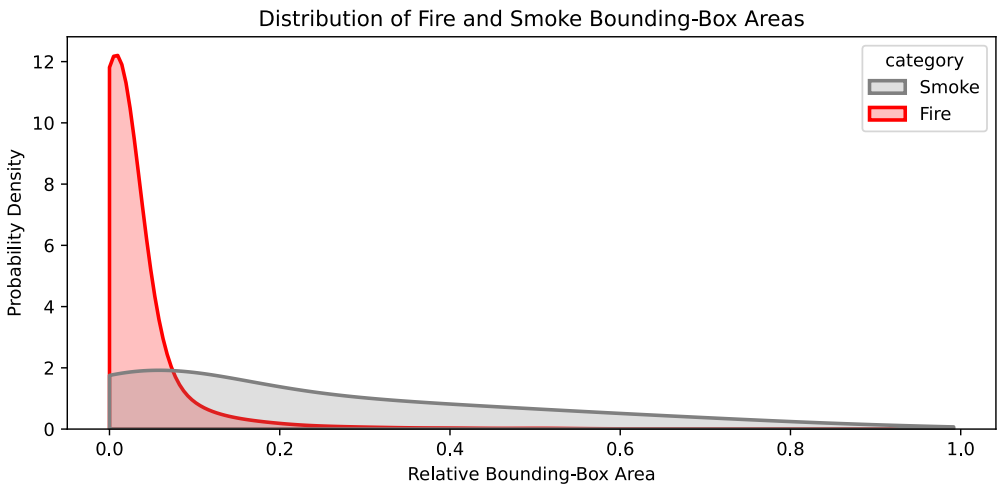


Figure 7: Kernel density estimation of relative bounding-box areas for fire and smoke annotations.

Finally, the distribution of bounding-box areas across the training, validation, and test splits was examined to determine whether each subset contains a representative range of bounding-box sizes. As shown in Figure 8, the distributions are broadly consistent across all three splits. Outliers were excluded from the visualisation to improve readability, allowing the general trends to be compared more clearly despite differences in the number of images in each split.

These findings suggest that no additional stratification or adjustments are necessary to ensure that bounding-box sizes are proportionally represented across the dataset splits.

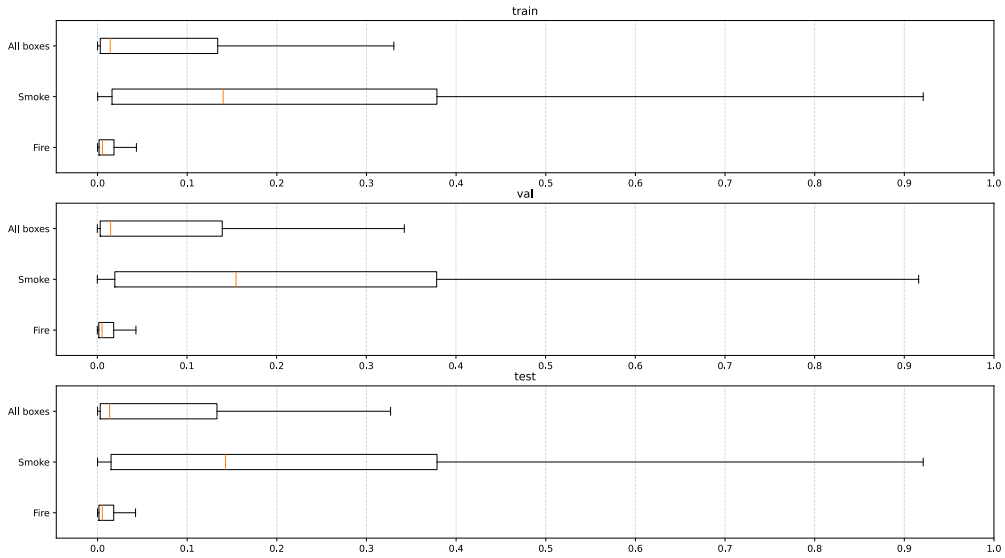


Figure 8: Box Plots of area distribution by split

1.2.3 Spatial Distribution of Bounding Boxes

The spatial locations of the bounding boxes were examined by plotting the distribution of their centre coordinates, as shown in Figure 9. Both fire and smoke annotations are concentrated towards the centre of their images, with smoke peaking slightly above the centre and fire spreading more evenly around it. Horizontally, both distributions are roughly symmetric, indicating no strong bias towards either side of an image. Small clusters of fire boxes can also be seen at the far left and right edges, which are likely objects cut off by the image boundaries.

This central concentration suggests that the model could learn to associate fire and smoke with the centre of an image rather than relying on their visual features, which is considered when selecting the augmentations in Section 2.4.

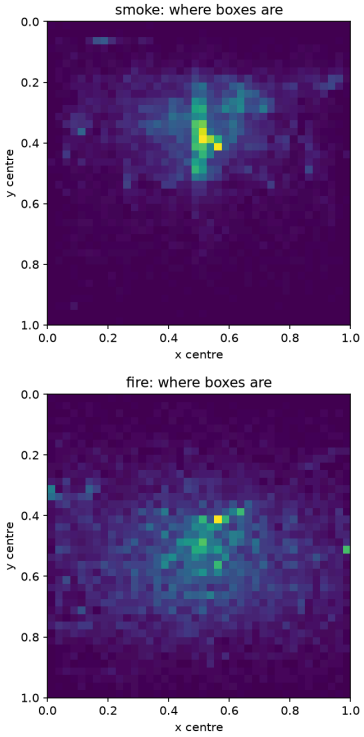


Figure 9: Distribution of bounding-box centres for smoke and fire

1.3 Image Properties

This section looks at how properties such as brightness, contrast, sharpness etc. are split across the images and if there are any distinctive features about either fire or smoke to help distinguish from their surroundings. Two aspects are investigated, how these properties differ between fire, smoke and neither images, and how fire and smoke regions differentiate from their surrounding backgrounds.

This analysis also aims to identify any biases that could influence the model's performance. For example, if fire or smoke consistently appear under lighting conditions where the model may learn to associate these conditions with the presence of the object rather than relying on its visual features. Such relations could increase the chances of false positive and/or negatives when the model encounter different lighting conditions.

1.3.1 Property Distributions

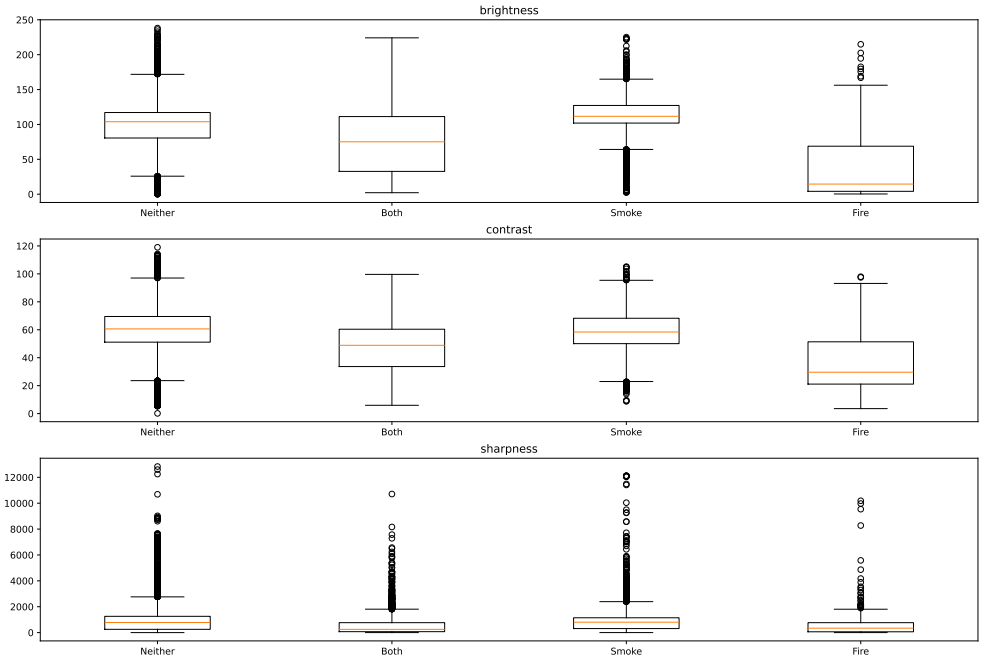


Figure 10: Property distribution across image types

All images are converted to gray scale and properties are calculated via the pixel values.

1.3.1.1 Brightness

Brightness is calculated as the mean pixel value across an image. Interestingly, between images that have no annotations and those that contain smoke there is a similar distribution while fire, which is something one would consider as bright, is seen as dark. This can be due to a multitude of reasons; the pictures are naturally darker, the exposure of the camera reducing the brightness of the fire to capture the rest of the image and, seen in Section 1.2.2, the fire boxes are generally tiny, so their brightness values across an entire image may not contribute as much to the overall mean, because it only takes up a small portion of the image. Further analysis will be done later to analyse any potential hypothesis for the reasons mentioned above.

1.3.1.2 Contrast

Contrast is calculated as the standard deviation of the pixel values across the image. The same trends can be seen as in Section 1.3.1.1. The reason for examining contrast is to understand how pixel values deviate from the mean image brightness and investigate whether the size of fire bounding boxes correlates with overall image contrast, particularly in cases where small fire regions occupy only a fraction of the image or larger fire regions contrast with darker backgrounds.

1.3.1.3 Sharpness

Sharpness is calculated as the variance of the Laplacian, which is the second order spatial differential operator of pixel intensities: $\text{Sharpness} = \frac{1}{N}\sum_{i=1}^{N}(L_i - \mu)^2$, where $L_i = \nabla^2 f(x, y)$ and $\mu = \text{avg}(L)$. This looks at how much pixel intensities change spatially, particularly around edges and fine details, but can also be influenced by the noise and texture of the image. Applying it to this case, it would generally be expected that smoke would have quite a low sharpness due to being highly diffuse, and fire to be quite high in sharpness. However, it also depends on the diffuseness of the

smoke. From Figure 10, smoke tends to have higher sharpness, but this can be due to reasons such as vegetation and other objects being visible through the smoke, meaning the image is not completely obscured. Also, contributions from other parts of the image can influence this. For similar reasons to those mentioned in Section 1.3.1.1, the edges of the fire may cause higher sharpness, but if most of the image is quite uniform, such as a wall, and the fire is small, the overall sharpness of the image would not be that high.

1.3.2 Properties Across Splits

The properties are then examined at the split-level in Figure 11 to ensure that no single split contains disproportionately bright images, images with high contrast, or images with high levels of detail as measured by sharpness, but rather that these properties are similarly distributed across the dataset splits. Significant differences between splits could introduce unintended biases, where the model may perform better on certain splits due to differences in image characteristics rather than its ability to detect fire or smoke. Therefore, maintaining similar distributions helps ensure that the validation and test sets provide a more representative evaluation of the model's performance.

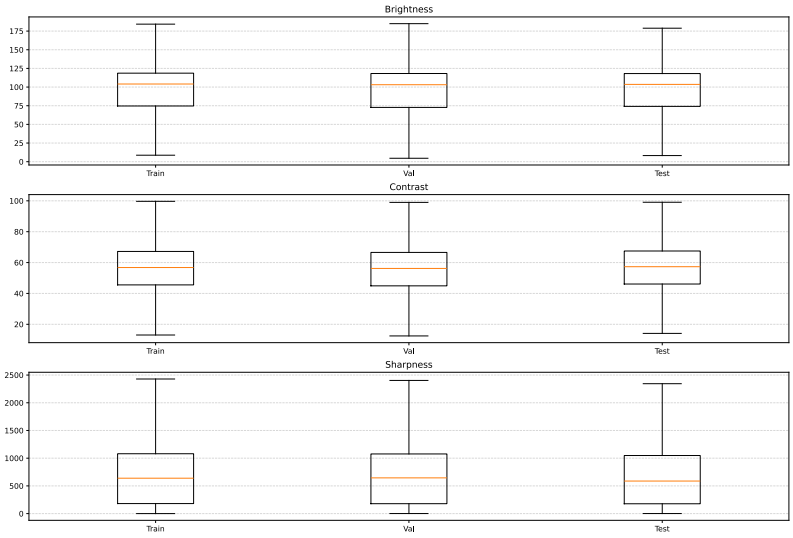


Figure 11: Image properties by split

1.3.3 Properties of Annotations

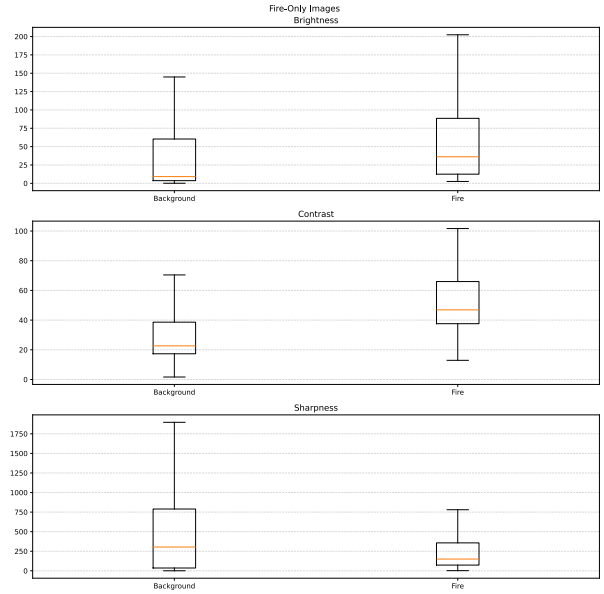


Figure 12: Fire boxes versus their background

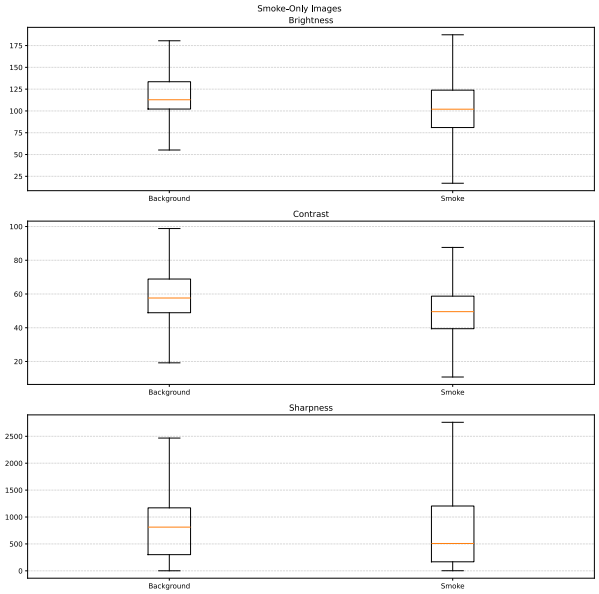


Figure 13: Smoke boxes versus their background

As discussed previously in Section 1.3.1.1 within those plots the properties were looked at a global image level, so it did not provide much information as to how these properties related to these bounding boxes. Properties within Figure 12 and Figure 13 are only looked at within a single class

type images due to the effects that terms like sharpness can have on boxes especially where fire and smoke may influence one another due to the nature of the items.

Within Figure 12 it is still seen that fire itself is generally quite dark where the image around the fire tends to be even darker. This raises the question of whether it is exposure, the time of day or the effect of tiny annotations seen in Section 1.2.2. This would require a lot more analysis into whether its actually night time, whether these small annotations are very far away or small fires which we believe to be beyond the scope of this project.

1.4 Correlations

This last section examines how different characteristics of the dataset relate to one another to check whether there are any linear or monotonic relationships within the dataset. For this analysis, we use Pearson's correlation coefficient to examine linear relationships, while Spearman's rank correlation coefficient is used to identify monotonic relationships. For example, this can include correlations between the number of bounding boxes and the size of an image, or how bounding-box coverage relates to global image properties such as brightness, contrast, and sharpness. Additionally, correlations between the properties of bounding boxes and their surrounding backgrounds can be examined, such as whether increases in bounding-box brightness are associated with changes in background brightness.

1.4.1 Box area compared to Box Count and Image Area

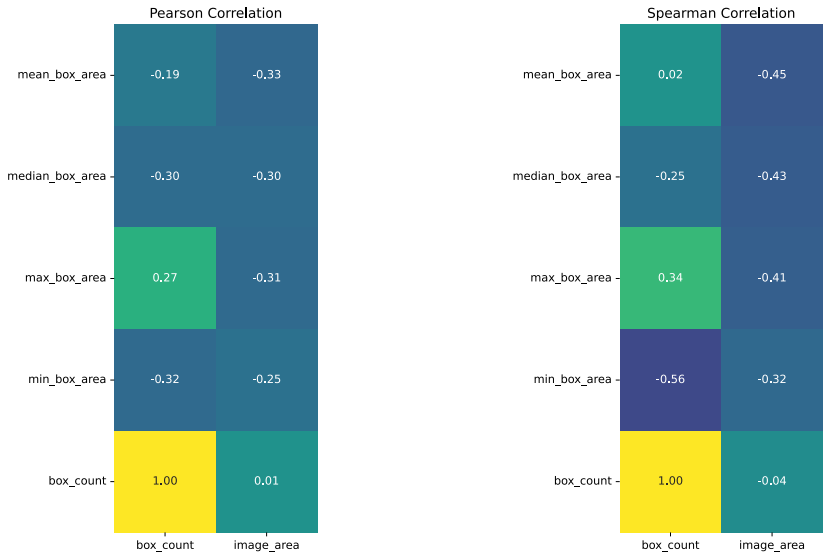


Figure 14: Correlation coefficients of box counts/image area vs box area

Box area is measured as the relative area of an image a box occupies in an image. The usefulness of Figure 14 is that it let us decide in preprocessing the downsizing of images. With the figure it can be seen that each box area tends to decrease when the image area gets larger which can be a problem when small boxes get compressed, leading to the loss of information. For example, they could be 20×20 pixels in their original image and could go as small as 2×2 pixels within the new image size for preprocessing.

Within box counts another relationship that can be seen is between the min and max areas. As the number of boxes increase, the minimum bounding box area tends to decrease, while the maximum area tends to increase. This could be due to differences in annotation practice, where some objects may be annotated using tiny bounding boxes or due to the nature of the images containing objects of various sizes. As discussed in Section 1.2.1, zero-area bounding boxes were removed during data cleaning. However, some miniscule annotations may still represent annotation errors or objects that

contain very little useful information after preprocessing. Determining whether these annotations are valid would require further manual inspection which is beyond the bandwidth of this project.

1.4.2 Annotations compared to their backgrounds

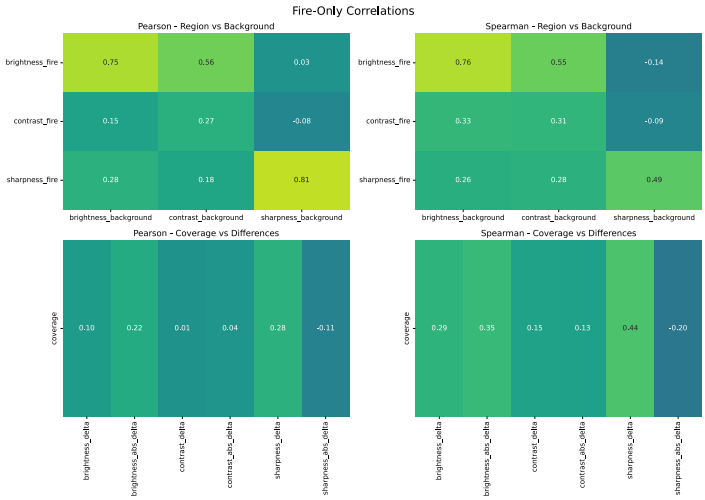


Figure 15: Correlation coefficients for fire-only boxes
(a) Bounding box properties vs global Properties
(b) Bounding box coverages vs global properties deltas

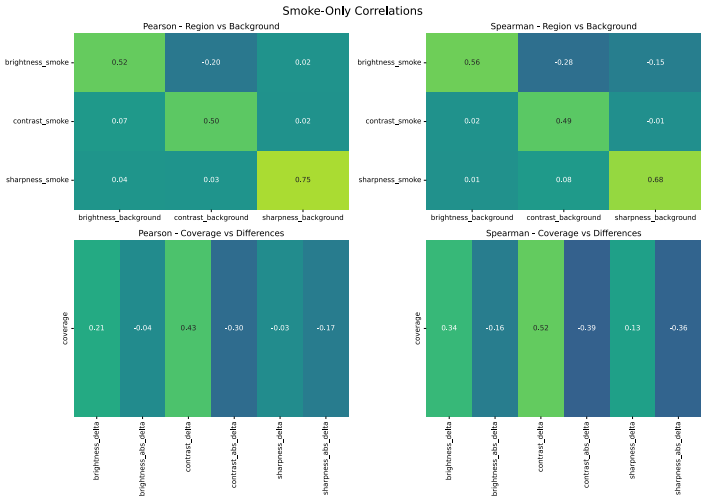


Figure 16: Correlation coefficients for smoke-only boxes
(a) Bounding box properties vs global Properties
(b) Bounding box coverages vs global properties deltas

Within Section 1.3.3 it was seen there was a gap between the bounding box annotation properties and the background of those images. Figure 15 and Figure 16 aim to help quantify these differences in seeing how strongly the properties of fire and smoke regions correlate with those of their surrounding backgrounds. They also examine the coverage of these bounding boxes and how their properties change relative to their background.

In Figure 15, there is a strong positive correlation between the brightness of fire bounding boxes and their surrounding backgrounds, with Pearson (0.75) and Spearman (0.76), respectively, indicating that both tend to increase and decrease together. As hypothesised in Section 1.3.3, one possible explanation is that many images were taken at night. However, this correlation alone cannot determine whether that is the case, as it could also be caused by fire illuminating its surroundings or bounding boxes not perfectly fitting the fire and including some background pixels.
Another possible explanation mentioned was camera exposure, which could influence the properties of the image depending on how the photograph was taken. A strong Pearson correlation of 0.81 is observed between fire and background sharpness, although this weakens to a moderate Spearman correlation of 0.49, potentially indicating that extreme values influence the Pearson coefficient. It was also hypothesised that brighter fires could cause overexposure or blur, reducing the sharpness of their surroundings. However, this is not strongly supported by the correlation between fire brightness and background sharpness, with Pearson (0.03) and Spearman (-0.14), indicating little to no consistent relationship across the dataset.
Coverage-wise, fire bounding boxes generally do not show strong relationships with differences in their background properties, with the most apparent relationship being sharpness, with Pearson (0.28) and Spearman (0.44), indicating a weak to moderate positive relationship. However, when examining absolute differences, these correlations weaken to the point where there is little to no apparent relationship. The difference measures how much a property increases or decreases between the bounding box and its background, while the absolute difference measures the magnitude of this change regardless of its direction. This suggests that greater fire coverage may be associated with

the direction of the sharpness difference, rather than necessarily producing larger differences in sharpness.

In Figure 16, most relationships are weak to moderate, except for sharpness, which has a Pearson correlation of 0.75 and a Spearman correlation of 0.68, indicating a moderate to strong positive relationship between smoke-region and background sharpness. However, this does not reveal much about the underlying cause, as the relationship could exist for several reasons, such as smoke being partially transparent and allowing background details to remain visible, or both regions sharing similar image characteristics. Therefore, while a strong relationship is observed, further analysis would be required to determine what contributes to it.

Coverage-wise, the only relationship of interest is contrast, with Pearson (0.43) and Spearman (0.52), indicating a moderate positive correlation between smoke coverage and the difference in contrast between smoke and its background. This suggests that as smoke occupies a larger proportion of an image, its contrast relative to the background tends to increase, which may be related to the varying density and transparency of smoke. As discussed previously regarding sharpness, these properties may also be influenced by the environment and conditions in which an image was captured, rather than smoke alone.

2 Preprocessing

From Section 1 analysis we decided on this preprocessing pipeline for our data. The data was cleaned for any mistakes in labelling discussed in Section 1.2.1

2.1 Resizing and Letterboxing

Letterboxing is a technique recommended by Ultralytics[2] as it helps maintain the aspect ratios and proportions of objects when images are resized. This introduces padding around images when their aspect ratios differ from the designated input dimensions. One potential problem is that smaller objects may lose further visual information during down scaling, particularly when their dimensions are already small. However, experimenting with and without letterboxing showed improvements in small-object detection, despite some reductions in overall model performance. Therefore, we decided to use letterboxing and resize all images to a fixed resolution of 768×768.

2.2 ColourJitter

We randomly augment the brightness, contrast, saturation and hue of images using torchvision's ColourJitter within the training set. This introduces variations in image properties during training, allowing the model to generalise better across different lighting conditions and environments. The selected values determine the range from which the augmentation factors are randomly sampled.

- **Brightness = 0.2 [0.8, 1.2]:** Randomly adjusts the brightness of an image. This is particularly useful for fire images, which were generally found to be darker during EDA. Randomly brightening or darkening these images can help reduce the model's reliance on specific lighting conditions.
- **Contrast = 0.2 [0.8, 1.2]:** Randomly adjusts the differences between brighter and darker regions of an image. This can help the model generalise across environments where fire and smoke may appear more or less distinguishable from their backgrounds.
- **Saturation = 0.2 [0.8, 1.2]:** Randomly adjusts the intensity of colours within an image. This helps account for differences in camera characteristics and environmental conditions, particularly where fire and smoke may appear with different colour intensities.

- **Hue = 0.015 [−0.015, 0.015]:** Randomly shifts the hue within torchvision's normalised hue range. This value is the default hue augmentation parameter used by Ultralytics YOLO [3]. The relatively small adjustment helps preserve the general colours of fire and smoke while introducing some colour variation.

2.3 Normalisation

The images are then normalised using the standard RGB channel statistics from ImageNet, which is the dataset used to pretrain our ResNet backbone. The mean values for these channels are [0.485, 0.456, 0.406], with standard deviations of [0.229, 0.224, 0.225] [4], respectively. Normalising using these values helps maintain consistency with the input distribution used during pretraining, allowing the model to make better use of its pretrained weights rather than having to adapt to an entirely different input scale.

2.4 Random Augmentations

Two random augmentations introduced into our pipeline to help improve the generalisation of our model were horizontal flips and random zoom-ins. Boundary boxes were also updated relative to whenever each augmentation was applied.

- **Horizontal Flips (p = 50%)** - This augmentation randomly flips images horizontally, helping prevent the model from becoming overly reliant on the horizontal positioning of objects and backgrounds. As shown in Section 1.2.3, fire and smoke are concentrated towards the centre of images and are roughly symmetric horizontally, so flipping does not change their overall positional distribution. Instead, it increases the variety of left/right arrangements of objects and backgrounds seen during training, encouraging the model to recognise fire and smoke under different spatial arrangements rather than associating them with particular sides of an image.
- **Random Zoom-ins (p = 50%)** - As discussed in Section 1.2.2, fire bounding boxes generally occupied a very small proportion of their images. To help address this, random zoom-ins were introduced to increase the apparent size of smaller annotations during training, potentially making them easier for the model to learn from. The maximum zoom-in factor was 30%, bounding boxes were clipped or removed if they were no longer in this new image.

3 Network Structure

Our network structure consists of a ResNet [5] backbone and a YOLOv8 [6] neck and head. The ResNet backbone is built layer-by-layer. The YOLOv8 neck is wired together by us from the Ultralytics Conv and C2f blocks, while the head is imported as a module from the Ultralytics library. The model also borrows its loss from the Ultralytics library.

We chose this model as, given the dataset size, it was useful to be able to initialise the ResNet backbone with ImageNet pretrained weights. We chose to combine this with a YOLO neck and head as it is suited to the wide range of object sizes in our images (both small fires and large smoke regions).

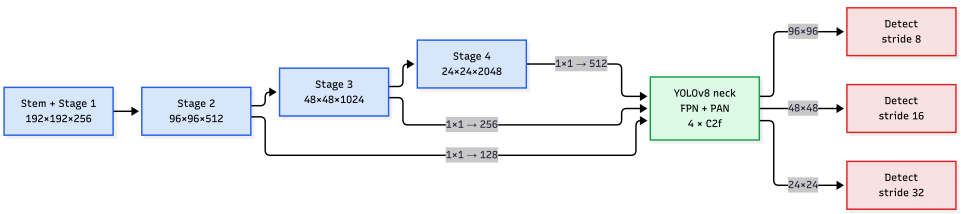


Figure 17: Our hybrid model

3.1 ResNet-50 Backbone

The backbone is ResNet-50 [5], implemented layer-by-layer in PyTorch. It is made up of a stem followed by four stages of residual blocks. We drop the average pooling at the end, since we only need the backbone to extract features, not to classify images.

The image goes through a 7×7 convolution with stride 2, batch normalisation and ReLU, and then a 3×3 max-pool with stride 2. By the time it reaches the residual stage, the image is 4 times smaller.

For the residual blocks, each has three convolutions; a 1×1 convolution that squeezes the channels down to a width w, a 3×3 convolution at that width, and another 1×1 convolution that brings the image back to $4w$ channels. Every convolution is followed by batch normalisation, with the first two also having ReLU. The output of the block is $\mathrm{ReLU}(F(x) + S(x))$, where S is the shortcut. If the input and output shapes match, the shortcut is just the identity. If they don't, we use a 1×1 convolution with batch normalisation to make them match.

The four stages have 3, 4, 6 and 3 blocks. The first block in each stage uses a projection shortcut to change the number of channels. In stages 2-4 it also halves the resolution.

Layer	Blocks	Output ($C \times H \times W$)	Stride
Input	–	$3 \times 768 \times 768$	1
Stem	–	$64 \times 192 \times 192$	4
Stage 1	3	$256 \times 192 \times 192$	4
Stage 2 (C_3)	4	$512 \times 96 \times 96$	8
Stage 3 (C_4)	6	$1024 \times 48 \times 48$	16
Stage 4 (C_5)	3	$2048 \times 24 \times 24$	32

Table 1: Feature map shapes through the ResNet-50 backbone for a 768×768 input.

We take the outputs of stages 2, 3 and 4 and pass them on to the neck. These are strides 8, 16 and 32, which line up with the 3 detection scales in YOLOv8. The finer maps help with small objects like distant fires, while the coarser ones are better for large patches of smoke.

3.2 Adapter

ResNet's channel counts are larger than the YOLO neck expects so to connect the backbone to the neck, we implemented an adapter. It consists of a 1×1 convolution, followed by batch normalisation and the SiLU activation function, which brings the channels down to widths $= (128, 256, 512)$ to match what the YOLO neck expects.

3.3 YOLOv8 Neck

In CNNs, the model shrinks the image as it goes deeper into the model which can result in early, large maps that know *where* objects are in the image but not *what* they are. Deeper layers with smaller maps will know what things are but only roughly where. For the model to learn better as a whole, we mimicked the architecture of modern YOLO models and used the FPN [7] top-down and PAN [8] bottom-up structure for the neck of the model.

The neck first completes a top-down pass, which upsamples smaller, deeper layers to match the size of the next, higher-resolution map (Figure 18). It then concatenates the two maps. Upsampling happens twice before the model moves to the PAN pass.

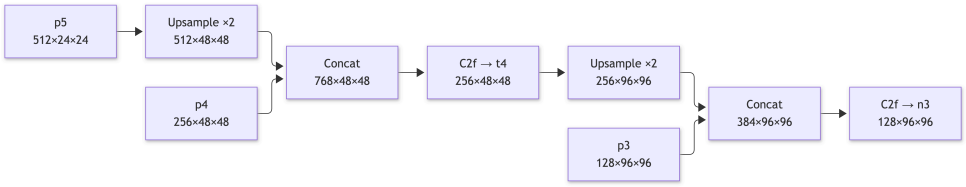


Figure 18: YOLOv8 neck: the top-down FPN pathway

A bottom-up pass is then completed, where the earlier, finer maps are downsampled to the size of the next, lower-resolution map and concatenated with the smaller map (Figure 19). This also happens twice before the model moves to the head.

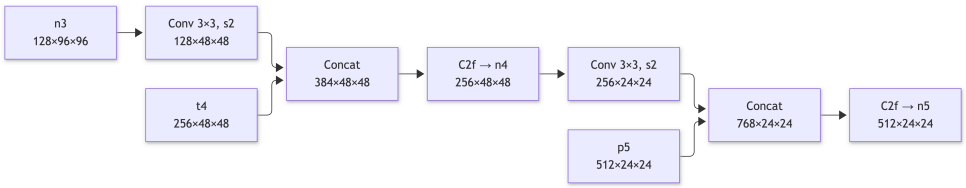


Figure 19: YOLOv8 neck: the bottom-up PAN pathway

3.4 YOLOv8 Head

The head of our model is a single Detect module imported from the Ultralytics library [6], which has a separate set of layers for each of the three feature maps coming from the neck of the model. There are two separate branches per feature map. The first is a classification branch, which gives a confidence score for each class and the other is a box branch which predicts how far the object's left, top, right and bottom edges are from the centre of each grid cell.

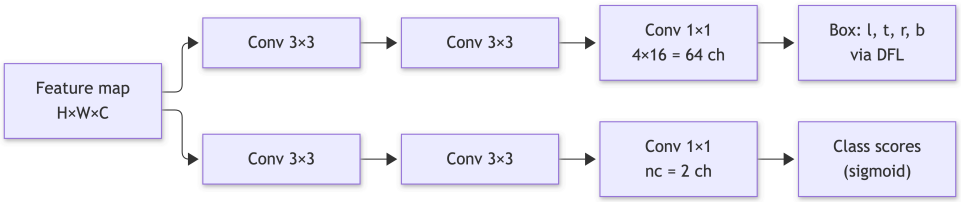


Figure 20: Detect head: separate box and classification branches per feature map.

3.5 Transfer Learning

Our ResNet-50 backbone is initialised with torchvision's 'IMAGENET1K_V2' weights [4], which were trained to classify the 1.28 million images of ImageNet into 1,000 classes. The backbone therefore already encodes general visual features such as edges, textures and shapes. Because our model only uses the backbone as a feature extractor, the final fully-connected classification layer is left out and only the stem and the four residual stages are used.

Component	Initialisation	Learning rate
ResNet-50 stem + stages 1–4	ImageNet-1K V2	$0.1 \times \eta$
1×1 adapter convolutions	Random	η
YOLOv8 neck (FPN + PAN)	Random	η
YOLOv8 detection head	Random	η

Table 2: Initial learning rate of each component.

No layers of our model are frozen. We opted to instead train the model on a differential learning rate that applies a learning rate 10 times smaller than the rest of the model to only the backbone. This

prevents the model from losing the feature knowledge it had already developed from training on ImageNet.

The 1×1 adapter convolutions, the YOLOv8 neck and head all borrow only the YOLOv8 architecture and have randomly initialised weights, therefore they are trained from scratch at a full learning rate.

3.5.1 Weight Initialisation

The randomly initialised layers use PyTorch's default Kaiming uniform initialisation, which keeps activations from shrinking or growing through the network. The classification biases in the head are set by Ultralytics [6] so that every class starts with a confidence close to 0. As most of the image is background, this stops the model from making large errors in the first epochs.

3.6 Residuals

Residuals appear in our ResNet-50 backbone, where each of the 16 bottleneck blocks has one residual connection. These connections make a deep network like ResNet-50 easier to train, because they help avoid the vanishing gradient problem [5].

3.6.1 The Vanishing Gradient Problem

During backpropagation, the gradient is passed backwards through the network one layer at a time, and is multiplied by each layer's weights along the way. In a deep network, these repeated multiplications can shrink the gradient to almost zero before it reaches the early layers. Those layers then barely update, and training slows down or stalls.

3.6.2 How Residual Learning Helps

In a normal block, the layers have to learn the full output $\mathcal{H}(x)$ from the input x. In a residual block, the layers only learn the change that needs to be made to the input, $\mathcal{F}(x) = \mathcal{H}(x) - x$. The input is then added back on using a shortcut connection:

$$y = \mathrm{ReLU}(\mathcal{F}(x) + x)$$

This is easier to learn, as each block only has to make a small change to the features it receives. If a block is not needed, its layers can just learn to output zero, and the input passes through unchanged.

The shortcut also helps with vanishing gradients. During backpropagation, the gradient can travel back through the shortcut without being multiplied by any weights. So even if the gradient through the layers becomes very small, the early layers still receive a useful gradient.

3.6.3 Residuals in Our Model

In our bottleneck blocks, the layers that learn $\mathcal{F}(x)$ are the three convolutions and their batch normalisations (Section 3.1). The input is added after the last batch normalisation, and the result is passed through ReLU. There are two types of shortcut:

- **Identity shortcut:** if the input and output are the same shape, the input is added on directly. This is used in 12 of the 16 blocks.
- **Projection shortcut:** the first block of each stage changes the number of channels, and in stages 2-4 also halves the image size. The input can no longer be added directly, so a 1×1 convolution and batch normalisation first reshape it to match. This is used in the other 4 blocks.

Our ImageNet pretrained weights were trained with these shortcuts, so we kept them in our model to make the most of transfer learning (Section 3.5).

3.7 Activation Functions

Our model used both ReLU and SiLU as activation functions. The ResNet backbone makes use of the ReLU while the Adapter, neck and head all use SiLU. The output of the head uses the sigmoid activation function for classification and softmax for the box regression values.

3.7.1 ReLU (backbone)

The ReLU activation is used after the stem convolution, after the first two convolutions of each bottleneck and after the residual addition. The ReLU activation function is defined as:

$$\mathrm{ReLU}(x) = \max(0, x) = \begin{cases} x \text{ if } x > 0 \\ 0 \text{ if } x \leq 0 \end{cases}$$

Our decision to use the ReLU activation function was based on the fact that the ResNet architecture was designed and pretrained using ReLU, along with our model's initialised ImageNet weights being learned under ReLU.

The main drawback of using ReLU is the problem known as "dying ReLU". The gradient of ReLU is 0 for negative inputs, so if a unit's input becomes negative for every image, it stops updating and always outputs 0. We use SiLU in the adapter, neck and head, so this problem does not affect the new layers.

3.7.2 SiLU (adapter, neck and head)

The SiLU activation function is used in the neck and head of our model. The SiLU function is defined as:

$$\mathrm{SiLU}(x) = x \cdot \sigma(x) = \frac{x}{1 + e^{-x}}$$

SiLU mitigates the "dying ReLU" problem by allowing for non-zero outputs for negative inputs, which helps to maintain gradient flow during training.

3.7.3 Sigmoid (head)

The sigmoid activation function is used in the head of our model for classification. The sigmoid function is defined as:

$$\mathrm{sigmoid}(x) = \frac{1}{1 + e^{-x}}$$

The sigmoid function outputs values between 0 and 1, which is suitable for binary classification tasks. In our case, it is applied to each class score separately, giving an independent probability that a predicted box contains fire and that it contains smoke.

3.7.4 Softmax (head)

The softmax activation function is used in the head of our model for box regression. The softmax function is defined as:

$$\mathrm{softmax}(x_i) = \frac{e^{x_i}}{\sum_{j=1}^{16} e^{x_j}}$$

Softmax turns a set of scores into probabilities that add up to 1. For each edge of a box, the head gives a score to 16 possible distances from the centre. Softmax turns these into probabilities, and the predicted distance is their weighted average.

3.8 Batch Normalisation

Batch normalisation is used heavily throughout our hybrid model, after almost every convolutional layer at all stages in the model. Batch normalisation is used to normalise the inputs to each layer to a mean of 0 and a standard deviation of 1. This helps to stabilise the training process and allows for higher learning rates, which can lead to faster convergence.

In the ResNet backbone, we use batch normalisation after the stem convolution, after all three convolutions of each bottleneck and after the 1×1 projection shortcut convolution. It is not used after the residual addition, which is only followed by ReLU. In the YOLOv8 neck, we use batch normalisation after every convolutional layer. In the YOLOv8 head, we use batch normalisation after every convolutional layer except for the final output layer, as it outputs the final predictions.

3.9 Regularisation

Rather than a single mechanism, our model is regularised through a combination of techniques, most of which are described in more detail in other sections:

- **Data augmentation:** the colour jitter (Section 2.2), horizontal flips and random zoom-ins (Section 2.4) are applied only to the training set and are re-sampled every epoch, so the model rarely sees the same image twice.
- **Weight decay:** AdamW [9] applies a weight decay of $\lambda = 10^{-4}$ to all parameters, shrinking the weights slightly at each step and penalising large weights.
- **Differential learning rate:** training the backbone at $0.1 \times \eta$ keeps it close to its pretrained ImageNet weights (Section 3.5), limiting how much it can overfit our much smaller dataset.
- **Early stopping:** a plateau is detected when a line fitted to the last 5 validation losses is no longer falling by more than 0.1% per epoch. On the first plateau, the learning rate is halved and then decayed with a cosine schedule, and training stops on the second plateau. The weights from the epoch with the best validation mAP50-95 are kept, so later overfitted epochs are never used.
- **Batch normalisation:** as the statistics in Section 3.8 are computed per mini-batch, they add a small amount of noise to the activations during training, which has a mild regularising effect.

3.10 Hyperparameters

The main hyperparameters of our final model are listed in Table 3. The learning rate, weight decay and backbone learning rate were chosen through hyperparameter tuning (Table 28), while the loss gains are the Ultralytics defaults [6].

Hyperparameter	Value
Input size	768×768
Batch size	8, with 4 gradient accumulation steps (32 images per update)
Epochs	Up to 50 (early stopping ended training after 27)
Optimiser	AdamW
Learning rate η	10^{-3}
Backbone learning rate	$0.1 \times \eta$
Weight decay λ	10^{-4}
Learning rate schedule	Halved on the first validation loss plateau, then cosine decay to $0.1 \times \frac{\eta}{2}$; training stops on the second plateau
Loss gains	Box 7.5, classification 0.5, DFL 1.5

Table 3: Main hyperparameters of the final model.

4 Loss Function, Optimiser and Cross-Fold Validation

4.1 Loss Function

Object detection is a multi-task problem. For every object, the model has to predict where it is (a bounding box) and what it is (fire or smoke). No single standard loss covers both, so we imported the YOLOv8 detection loss [6], which is a weighted sum of three terms:

$$L = \lambda_{\text{box}} L_{\text{CIoU}} + \lambda_{\text{cls}} L_{\text{BCE}} + \lambda_{\text{dfl}} L_{\text{DFL}}$$

Before the loss is calculated, each ground-truth box is matched to the predictions that best fit it, using the Task-Aligned Assigner. All other predictions are treated as background, so images with no boxes still teach the model what background looks like.

The three terms are:

- **CIoU box loss**: penalises low overlap between the predicted and true box, distance between their centres, and different aspect ratios. Overlap is what mAP uses to judge a detection, and the aspect ratio term helps with smoke, whose boxes vary a lot in shape (Figure 13).
- **Binary cross-entropy (BCE) classification loss:** scores each class separately, so a region can be both fire and smoke, which often happens in our images.
- **Distribution Focal Loss (DFL)**: instead of predicting one exact position for each box edge, the model predicts a range of likely positions. This suits smoke, whose edges are blurry.

The gains λ balance the three terms. The box term has the highest gain because localisation is the harder part of the task. We used the Ultralytics defaults (Table 3) for gains values.

4.1.1 Alternative Loss Functions

We considered alternative loss functions, but opted not to use them for the following reasons:

- **L1 / L2 on box coordinates**: depend on box size and do not directly improve overlap.
- **IoU, GIoU and DIoU:** IoU gives no gradient when boxes do not overlap; GIoU and DIoU fix this, but lack CIoU's aspect ratio term.
- **Softmax cross-entropy:** treats fire and smoke as mutually exclusive.
- **Focal loss**: handles the large number of background predictions, but the Task-Aligned Assigner already reduces this problem.

4.2 Optimiser

We used the AdamW optimiser [9]. It keeps a running average of recent gradients and of their squares, and uses these to give every weight its own step size. Weights with large, noisy gradients take smaller steps, while weights with small gradients take larger ones.

Our model combines a pretrained backbone with newly initialised layers, and the loss combines three terms with different gains, so gradient sizes vary a lot across the network. AdamW's per-weight step sizes handle this without us having to tune each part separately. It also converges quickly with little tuning, which was important because each configuration had to be trained five times for cross-validation.

The learning rates and weight decay used with AdamW were chosen through hyperparameter tuning and are listed in Table 3.

4.2.1 Alternative Optimisers

Some other optimisers we considered were:

- **SGD with momentum:** commonly used to train ResNet and YOLO, but is more sensitive to the learning rate and usually needs longer training.

- **Adam:** weight decay is weakened by the adaptive step size, which would give less reliable regularisation.
- **RMSProp:** adapts step sizes like Adam but has no momentum, so updates would be noisier.

4.3 Cross-Validation

Some of the images in the dataset are near-identical frames of the same scene. If one of these is in the training fold and another in the validation fold, the model is tested on an image it has already seen which can cause data leakage. To avoid this, each image is given a difference hash (dHash) [10], and images whose hashes differ by 4 bits or less are placed in the same group. The groups are then split across five folds, so a group is never in two folds [11]. The original splits do not do this, as 1,459 of the 3,099 validation images share a group with a training image.

Fold	Images	Groups	Empty images	Smoke boxes	Fire boxes
0	3,445	1,922	47.5%	1,901	2,095
1	3,444	1,922	45.0%	2,002	2,134
2	3,444	1,925	38.2%	2,074	3,017
3	3,444	1,923	55.6%	1,611	2,212
4	3,444	1,926	41.1%	1,955	2,349

Table 4: Composition of each cross-validation fold.

The folds have the same size but differ in content (Table 4). Fold 2 has the most fire boxes and the fewest empty images while fold 3 has the most empty images.

To keep training time down, each run trains for 8 epochs on 3,000 images from four folds and is validated on 1,000 images from the fifth. The other settings match the full training (batch size of 8 with 4 accumulation steps, 768 pixel images, AdamW optimiser [9]). One epoch took about 97 seconds on an RTX 3080 (10GiB VRAM variant).

We cross-validated the base setting and a weight decay of 10^{-3}, the only change that beat the base in hyperparameter tuning (Table 28).

Fold	Base	Weight decay 10^{-3}	Difference
0	0.112	0.124	+0.012
1	0.112	0.113	+0.001
2	**0.151**	0.141	−0.011
3	0.125	0.121	−0.005
4	0.133	0.137	+0.005
Mean ± std	**0.127 ± 0.017**	**0.127 ± 0.012**	+0.0005
mAP50 mean	0.336	0.336	0.000
recall50 mean	0.861	0.856	−0.005
mAR100 mean	0.431	0.428	−0.003

Table 5: Best validation mAP50-95 on each fold, with means of other metrics.

Both settings perform the same (Table 5). Their mean mAP50-95 differs by only 0.0005 and a paired t-test [12] gives $p = 0.90$. Weight decay 10^{-3} is furthest ahead at fold 0, while the base is furthest ahead on fold 2.

Both settings score highest on fold 2, which has the most fire boxes and the fewest empty images, and lowest on fold 1. In 9 of the 10 runs the best epoch was the last one so the model was still

improving when training stopped (Figure 21). Training for longer likely has more influence than the choice of weight decay.

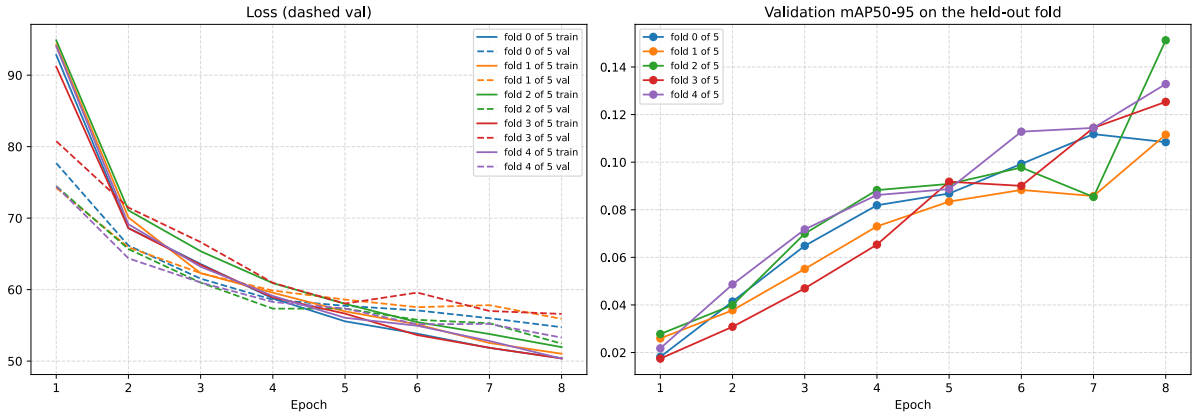


Figure 21: Training (solid) and validation (dashed) loss, and validation mAP50-95, for each fold of the base setting.

4.4 Model Comparison

For each predicted box, we compare it to the true box with intersection over union (IoU), the area where the two boxes overlap divided by the total area they cover together.

$$\text{IoU} = \frac{|B_{\text{pred}} \cap B_{\text{true}}|}{|B_{\text{pred}} \cup B_{\text{true}}|}$$

IoU ranges from 0 to 1, in relation of when the boxes do not overlap to when they match exactly. A prediction counts as correct if the IoU with a true box reaches a threshold, a higher threshold means the box fits more tightly.

All models are scored with COCO average precision from torchmetrics [13]. Before scoring, overlapping detections are removed with non-maximum suppression, using the settings Ultralytics validates with (confidence 0.001, IoU 0.7, at most 300 boxes per image).

Metric	Meaning
mAP50-95	Average precision over IoU thresholds 0.50 to 0.95, averaged between both classes
mAP50	Average precision at IoU 0.50
recall50	Share of boxes found at IoU 0.50
mAR100	Recall averaged over IoU 0.50 to 0.95, up to 100 detections per image
Small / medium / large	Same metrics for boxes under 32^2, between 32^2 and 96^2, and over 96^2 pixels

Table 6: Metrics used to compare models.

mAP50-95 is our main metric, as it rewards boxes that are both correct and tight. It is used to pick the best epoch of each run and to compare runs. For a fire detector, the consequences of missing a fire are far greater than a false alarm, so we also check recall, including for small boxes, as most fire boxes are small.

During tuning, every run uses the same seed and the same images, so only the setting being tested changes. We treat differences under 0.01 mAP50-95 as noise and settings that beat the base by more than that are cross-validated.

5 Results

This section evaluates the performance of our model on the test set and compares its results with those reported in the literature using similar datasets and detection architectures. Finally, it investigates the presence of near-duplicate images across the dataset splits and examines how this overlap affects the reported evaluation metrics and the reliability of the model's measured generalisation performance.

5.1 Model Results

The model with hyperparameters mentioend in Section 3.10 performed well throughout training, as shown in Figure 22. However, there are indications of potential overfitting, as training loss continued to decrease while validation loss began to plateau, resulting in an increasing gap between the two. Despite this divergence, validation mAP metrics continued to improve, suggesting that the model was still improving its detection performance even though the validation loss showed little further reduction. This highlights that improvements in detection performance do not necessarily correspond directly to reductions in validation loss. This discrepancy could be attributed to differences between the YOLO loss function and mAP evaluation. While YOLO loss combines multiple optimisation objectives, including bounding-box regression and classification, mAP measures detection performance through confidence rankings and bounding-box overlap at different IoU thresholds. Consequently, improvements in detection performance may not correspond directly to reductions in the combined loss.

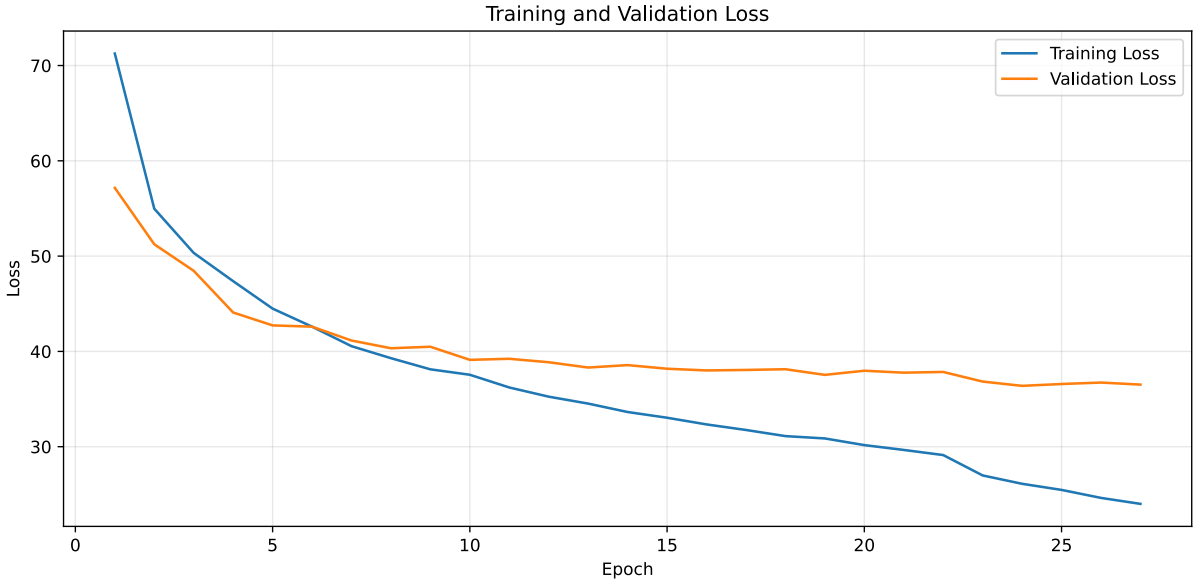


Figure 22: Training and validation loss across epochs.

On the test set, our model achieved an mAP50 of 0.7414 and an mAP50-95 of 0.4070, as shown in Table 7. The difference between these metrics indicates that while the model can detect and localise a reasonable proportion of objects at an IoU threshold of 0.50, performance decreases when stricter localisation requirements are introduced. This suggests that precise bounding-box localisation remains an area for improvement.

Metric	Score
mAP50	0.7414
mAP50-95	0.4070

Table 7: Overall detection performance on the test set.

Object size	mAP50-95	mAR	Recall50	Recall75
Overall	0.4070	0.5492	0.9042	0.5614
Small	0.2708	0.4104	0.8272	0.3429
Medium	0.4044	0.5480	0.9205	0.5670
Large	0.4727	0.6064	0.9248	0.6530

Table 8: Test-set detection performance by object size. Recall50 and Recall75 use IoU thresholds of 0.50 and 0.75, respectively.

As shown in Table 8, detection performance improves with object size across all four metrics. Small objects achieved the lowest mAP50-95 of 0.2708, compared with 0.4044 for medium objects and 0.4727 for large objects. This reinforces the concerns identified during the experiments regarding the difficulty of detecting smaller objects.

A particularly significant difference can be observed in recall. For small objects, recall decreases from 0.8272 at IoU 0.50 to 0.3429 at IoU 0.75, a reduction of 48.43 percentage points. In comparison, large-object recall decreases from 0.9248 to 0.6530, a reduction of 27.18 percentage points. This suggests that while the model can identify many smaller objects at lower IoU thresholds, achieving sufficiently accurate bounding-box localisation becomes considerably more difficult as the IoU threshold increases.

Class	mAP50-95
Smoke	0.4689
Fire	0.3451

Table 9: Test-set average precision for each class, averaged over IoU thresholds from 0.50 to 0.95.

As shown in Table 9, smoke achieved a higher mAP50-95 of 0.4689 compared with 0.3451 for fire, representing a difference of 12.38 percentage points. This indicates that the model performs considerably better at detecting smoke than fire. Although the aggregate metrics do not establish the cause of this difference, the results identify fire detection as an area requiring further improvement.

5.2 Results from Other Models

This section compares our model's performance against results reported in other research using the same underlying dataset or closely related subsets. The dataset used in this project was obtained from the Smoke-Fire-Detection-YOLO dataset on Kaggle, which is also distributed as a version of the D-Fire dataset.

Two papers were selected for comparison:

- **Cross-Dataset Evaluation of YOLOv8 for Unmanned Aerial Vehicle Fire and Smoke Detection** [14], which evaluates multiple YOLOv8 architectures on the D-Fire dataset.
- **TFNet: Transformer-Based Multi-Scale Feature Fusion Forest Fire Image Detection Network** [15], which investigates a specialised architecture combining a modified ResNet backbone, multi-scale feature fusion and transformer-based detection.

These studies were selected to compare our custom detection architecture against both established YOLO models and an alternative architecture designed specifically for fire and smoke detection. Differences in training data, dataset splits and experimental configurations are also considered when discussing potential reasons for their higher performance.

5.2.1 YOLOv8 Cross-Dataset Evaluation

The first paper, [14], evaluates three standard YOLOv8 architectures: YOLOv8n, YOLOv8s and YOLOv8m. Unlike our model, which uses an ImageNet-pretrained ResNet backbone with a randomly initialised YOLO-style neck and detection head, these models were initialised using COCO-pretrained object detection checkpoints. Consequently, their backbones, necks and detection heads had already learned features relevant to object detection before being fine-tuned on fire and smoke images.

The YOLOv8 architectures also use established multi-scale feature extraction and fusion to combine information from different spatial resolutions. This is particularly relevant to our model, as the results in Table 8 identified small-object localisation as an area requiring further improvement.

The paper reports the following results on the D-Fire test set:

Model	mAP50	mAP50-95
Our Model	0.7414	0.4070
YOLOv8n	0.7765	0.4489
YOLOv8s	0.7830	0.4524
YOLOv8m	0.7904	0.4638

Table 10: Comparison between our model and the YOLOv8 results reported by [14].

All three YOLOv8 variants achieved higher mAP50 and mAP50-95 scores than our model. Interestingly, the difference between the smallest and largest YOLOv8 variants was relatively small, with mAP50-95 increasing from 0.4489 to 0.4638. This suggests that increasing model capacity alone did not produce substantial improvements in detection performance.

One potential explanation for the higher scores is the difference in pretraining. Our model required the detection neck and head to learn from random initialisation, whereas the YOLOv8 models benefited from pretrained object detection weights. Furthermore, the paper used a training configuration of up to 100 epochs with a broader augmentation pipeline, including mosaic augmentation, compared with our shorter training runs.

However, the study used the official D-Fire training and testing partitions, while our project used a different selection of images and dataset split. Therefore, the results provide a useful comparison of model performance but cannot establish that the architectural differences alone caused the higher scores.

5.2.2 TFNet: Transformer-Based Multi-Scale Feature Fusion

The second paper, [15], proposes TFNet, a specialised fire and smoke detection architecture based on Real-Time Detection Transformer(RT-DETR). Unlike our model's conventional convolutional backbone and YOLO-style detection architecture, TFNet introduces several components designed to improve feature extraction, multi-scale feature fusion and bounding-box localisation.

Their first modification is the SRModule, which replaces parts of a ResNet18 backbone with specialised convolutional operations to extract a more diverse range of features. The second is the CG-MSFF Encoder, which combines adaptive feature fusion with attention mechanisms to better integrate information from different feature-map resolutions. Finally, TFNet uses a transformer-based decoder to refine object predictions and Wise-IoU loss to improve bounding-box regression.

Model	mAP50	mAP50-95
Our Model	0.7414	0.4070
TFNet	0.8120	0.4680

Table 11: Comparison between our model and TFNet, as reported by [15].

The higher performance of TFNet could be partly attributed to its specialised feature-fusion architecture. In particular, the attention-based combination of features at different resolutions may help preserve information about small fire and smoke objects, which our model struggled to localise accurately. Additionally, Wise-IoU modifies the contribution of bounding-box regression errors during training, potentially improving localisation performance.

However, TFNet was trained on a selected subset of 9,869 forest-related images from D-Fire, using a 70/20/10 split and 200 training epochs. These differences in dataset composition and training methodology mean that its results are not directly equivalent to those obtained by our model.

Overall, both papers demonstrate that established object detection architectures can achieve higher performance on related fire and smoke detection tasks. The YOLOv8 results highlight the potential benefits of detection-specific pretraining and established multi-scale architectures, while TFNet demonstrates how specialised feature fusion and bounding-box regression methods can further improve detection performance. These approaches provide potential directions for improving our model, particularly its performance on smaller objects.

5.3 Near-Duplicates in the Test Split

Using the same hash as in cross-validation, we see that 2,109 of the 4,306 test images (49%) have a near-duplicate in the train or validation split. These make the test score higher than it would be on new images, so Table 12 scores them separately.

Test images	Images	mAP50	mAP50-95	Smoke / Fire mAP50-95	mAR100
All	4,306	0.741	0.407	0.469 / 0.345	0.549
No near-duplicate	2,197	0.699	0.378	0.463 / 0.292	0.533
With near-duplicate	2,109	0.859	0.504	0.491 / 0.518	0.604

Table 12: Test results with and without near-duplicates of train or validation images.

Without near-duplicates, mAP50-95 drops from 0.407 to 0.378. The drop is mostly in fire, which falls from 0.345 to 0.292, while smoke only falls from 0.469 to 0.463. The validation split has the same problem, so the validation and test scores end up having similar results.

6 Independent Experiments

This section examines how different augmentations, image resolutions and regularisation techniques affect model performance during training. The aim is to identify which configurations improve performance and should be carried forward into the final experiments discussed in Section 6.8.

Hyperparameters such as learning rate and weight decay are kept constant across experiments unless they are the specific variable being investigated. This helps maintain consistency between training runs, allowing the effects of individual changes to be compared.

Results across experiments are compared using only the best values achieved on the validation set. Comparing metrics from every epoch would considerably expand the discussion without necessarily providing additional insight beyond the results already presented.

6.1 Baseline Run

The baseline run was conducted to establish an initial performance benchmark for the model, allowing subsequent experiments to be compared against it. The model was trained using the following parameters:

Image Size	Batch Size	Accum Steps
640×640	14	2

Table 13: Training parameters for the baseline experiment.

After 10 epochs, the model achieved the following results:

Epochs	Best mAP50	Best mAP50-95
10	0.657	0.335

Table 14: Detection performance after 10 epochs of baseline training.

These results give an overview of the initial performance of the model, with an mAP50 of 0.657 indicating reasonable detection performance at an IoU threshold of 0.5. However, the lower mAP50-95 of 0.335 suggests that performance decreases at stricter IoU thresholds, potentially indicating difficulties with precise bounding-box localisation. Further experimentation will examine how performance changes across these thresholds. As mentioned in Section 4.4, recall will also be used to evaluate detection performance across different bounding-box sizes, particularly smaller annotations.

6.2 LR Plateau Reductions

Due to our model starting from a pretrained backbone with a reduced learning rate, as mentioned in Section 3.5, the initial learning rate may not remain suitable throughout training. After several epochs, once the randomly initialised YOLO neck and head have begun learning, a smaller learning rate may help the model converge towards a better local minimum.

To identify when the learning rate should be reduced, a custom plateau detector was implemented using linear regression over a moving window of the five most recent validation losses. The slope is normalised by the absolute mean loss of the window, calculated as $m_{\text{relative}} = \frac{m}{|\mu_L| + \varepsilon}$. A plateau is detected when the relative slope exceeds -0.001, indicating that the validation loss is improving by less than approximately 0.1% per epoch or has begun increasing. The learning rate is then halved across all parameter groups in the optimiser.

This approach was preferred over a conventional fixed-delta detector because regression examines the overall trend across several epochs rather than individual improvements. Furthermore, normalising the slope makes the threshold less dependent on the magnitude of the validation loss, allowing the same criterion to be applied across different loss scales without requiring as much manual adjustment.

The parameters of this training run were as follows:

Image Size	Batch Size	Accum Steps
640×640	14	2

Table 15: Training parameters for the learning-rate plateau experiment.

The effects of the plateau scheduler were examined by comparing the best performance achieved before and after the learning-rate reduction. Epoch 10 was the final epoch before the reduction, while epoch 11 was the first epoch evaluated after it.

Epoch	Best mAP50	Best mAP50-95
10	0.669	0.342
11	0.688	0.363

Table 16: Best detection performance before and after the learning-rate reduction.

The model achieved its best metrics at epoch 11, immediately following the learning-rate reduction. This represented an improvement of 0.019 in mAP50 and 0.021 in mAP50-95 over the previous best results. However, no further improvements were achieved during the remaining training, suggesting that while the initial plateau reduction was beneficial, its effectiveness was limited beyond this point.

Due to the improvements observed following the plateau reduction, this approach was retained for subsequent experiments and used as the new baseline for comparison. Later experiments will investigate whether alternative learning-rate scheduling strategies can improve the model's performance further.

6.3 Random Horizontal Flips

Random horizontal flips were added to the augmentation pipeline to investigate whether changing the orientation of an image could improve model performance. As mentioned in Section 2.4, horizontal flipping has a 50% probability of being applied to each image during training, potentially allowing the model to generalise better to different spatial arrangements of fire and smoke annotations.

The parameters of the run were as follows:

Image Size	Batch Size	Accum Steps
640×640	14	2

Table 17: Training parameters for the random horizontal flip experiment.

The experiment achieved the following results, including the best small-object mAP50-95, which evaluates objects with an area smaller than 32^2 pixels, as mentioned in Section 4.4:

Epochs	Best Epoch	Best mAP50	Best mAP50-95	Best Small mAP50-95
18	16	0.721	0.391	0.227

Table 18: Detection performance with random horizontal flipping.

Adding random horizontal flipping improved the results compared with the previous experiment in Section 6.2, which achieved a best mAP50-95 of 0.363. This represents an improvement of 2.8 percentage points, in mAP50-95. The improvement suggests that introducing spatial variation through horizontal flipping can help the model generalise better to different object orientations and positions. Due to the improvement observed, this augmentation was selected for inclusion in the final training configuration. Despite the improvement in overall performance, small-object mAP50-95 remained considerably lower at 0.227 compared with the overall mAP50-95 of 0.391. This reinforces the concerns identified during EDA regarding the large number of small fire annotations and suggests that further experimentation should investigate how preprocessing and augmentation affect small-object detection.

6.4 Increasing the Resolution

There is a concern regarding the performance of small bounding boxes in the dataset. As observed in Section 6.3, small objects achieved significantly worse performance compared with the overall

average. This experiment investigates whether increasing the image resolution can improve the detection of these smaller objects.

The parameters for this run were as follows:

Image Size	Batch Size	Accum Steps
768×768	8	4

Table 19: Training parameters for the increased-resolution experiment.

The results were as follows:

Epochs	Best Epoch	Best mAP50	Best mAP50-95	Best Small mAP50-95
17	14	0.687	0.363	0.242

Table 20: Detection performance with an input resolution of 768×768.

Although random horizontal flipping was removed from this experiment, the increased resolution still achieved a higher small-object mAP50-95 of 0.242 compared with 0.227 in Section 6.3. This is particularly interesting given that horizontal flipping had previously improved overall model performance. While the experiment does not isolate the effect of resolution, the improvement in small-object detection provides justification for continuing with the higher resolution in subsequent experiments.

Recall was also recorded at different IoU thresholds to investigate how detection performance changes as the required bounding-box overlap becomes stricter. At epoch 14, small-object recall decreased from 0.784 at IoU 0.50 to 0.325 at IoU 0.75, indicating that precise localisation of small objects remains a significant challenge.

6.5 Cosine Annealing

As mentioned at the end of the discussion in Section 6.2, it was hypothesised that introducing a learning-rate scheduler could further improve model performance following the initial plateau reduction. By gradually decreasing the learning rate over subsequent epochs, the scheduler may allow the model to make smaller parameter updates and stabilise training during its final stages.

For example, examining the final epochs of the increased-resolution experiment in Section 6.4, Table 21 shows that following the learning-rate reduction at epoch 13, subsequent epochs produced only marginal improvements or decreases in mAP50-95. This motivated investigating whether further reductions in the learning rate through cosine annealing could improve the model's performance.

Epoch	mAP50-95
13	0.361
14	0.363
15	0.359
16	0.361
17	0.358

Table 21: Validation mAP50-95 during the final epochs of the increased-resolution experiment.

The parameters of this training run were as follows:

Image Size	Batch Size	Accum Steps
768×768	8	4

Table 22: Training parameters for the cosine annealing experiment.

Results shown as:

Epochs	Best Epoch	Best mAP50	Best mAP50-95	Best Small mAP50-95
15	15	0.700	0.373	0.242

Table 23: Detection performance with Cosine Annealing.

Cosine annealing was selected as the learning-rate scheduler, although exponential decay was also considered. Since the training process already incorporates a plateau-based learning-rate reduction, cosine annealing was preferred because it allows the learning rate to decrease smoothly following the initial reduction. The plateau detector determines when to begin reducing the learning rate, while cosine annealing gradually decreases it over the remaining epochs, allowing smaller parameter updates as training progresses.

The scheduler is activated when the first plateau is detected, at which point the learning rate is halved across all optimiser parameter groups. Cosine annealing is then applied over subsequent training epochs, gradually decreasing the learning rate to a tenth of the current learning rate over 8 epochs, according to a cosine-shaped curve. This allows the model to make progressively smaller parameter updates as training continues. The scheduler remains active until training terminates, either through early stopping or reaching the maximum number of epochs.

Comparing the results with the increased-resolution experiment in Section 6.4, introducing cosine annealing improved the best mAP50-95 from 0.363 to 0.373, representing an increase of 1 percentage point. The best mAP50 also increased from 0.687 to 0.700, while small-object mAP50-95 remained approximately unchanged at 0.242. This suggests that the improvement was primarily in overall detection performance rather than small-object detection.

The model achieved its best overall mAP50-95 at epoch 15, compared with epoch 14 in the previous experiment. These results support the hypothesis that further reducing the learning rate after the initial plateau reduction can improve model performance. Consequently, cosine annealing was retained for the final experiment.

6.6 Random Zoom-ins

The final independent experiment investigated the use of random zoom-ins. This builds upon the increased-resolution experiment in Section 6.4, where improvements in small-object detection were observed. As discussed in Section 2.4, random zoom-ins were applied with a probability of 50%, selecting a random region of the image with a maximum zoom of 30%. Bounding boxes extending beyond the selected region were clipped, while those that no longer intersected the region were removed.

The training parameters were as follows:

Image Size	Batch Size	Accum Steps
768×768	8	4

Table 24: Training parameters for the random zoom-ins experiment.

The results were as follows:

Epochs	Best Epoch	Best mAP50	Best mAP50-95	Best Small mAP50-95
18	17	0.721	0.388	0.243

Table 25: Detection performance with random zoom-ins.

In terms of overall performance, random zoom-ins improved the best mAP50-95 from 0.363 in Section 6.4 to 0.388, representing an increase of 2.5 percentage points. This is similar to the improvement observed when introducing random horizontal flipping in Section 6.3, suggesting that random spatial augmentations can be beneficial for overall detection performance.

However, the primary motivation for introducing random zoom-ins was to improve small-object detection. In this regard, the improvement was considerably smaller, with small-object mAP50-95 increasing from 0.242 to 0.243, a difference of just 0.1 percentage points. Such a small improvement could be attributed to variability in the training process rather than a meaningful improvement in small-object detection.

One potential explanation is that the zoom-ins were applied to randomly selected regions without considering the locations of annotated objects. Consequently, there was no guarantee that the selected region would contain a small object. A potential improvement would be to introduce object-aware zooming, where the selected region is influenced by the locations and density of bounding boxes. For example, the augmentation could target regions around the mean coordinates of annotated objects or bias the selection towards smaller bounding boxes, increasing the likelihood that small objects are enlarged during training.

Despite the limited improvement in small-object mAP50-95, a more significant improvement was observed in small-object recall. Compared with Section 6.4, small-object recall at IoU 0.50 increased from 0.787 to 0.850, an improvement of 6.3 percentage points. Similarly, recall at IoU 0.75 increased from 0.344 to 0.399, an improvement of 5.5 percentage points. This suggests that random zoom-ins improved the model's ability to detect small objects, even though the improvement in average precision was relatively small. However, the considerable difference between recall at IoU 0.50 and 0.75 indicates that precise localisation of small objects remains a challenge.

However, due to the improvement observed in overall detection performance, random zoom-ins were retained for the final training configuration.

6.7 Final Training Run

This final training run combines the configurations investigated in the previous experiments. While not all experiments were strictly independent, certain changes, such as the learning-rate plateau reduction in Section 6.2 and the increased image resolution in Section 6.4, demonstrated sufficient improvements to be retained in subsequent experiments. This final run investigates how combining random horizontal flips from Section 6.3, cosine annealing from Section 6.5, and random zoom-ins from Section 6.6 affects model performance. The resulting configuration will serve as the baseline for subsequent hyperparameter tuning.

The parameters of this run were as follows:

Image Size	Batch Size	Accum Steps
768×768	8	4

Table 26: Training parameters for the final experiment.

The results were as follows:

Epochs	Best Epoch	Best mAP50	Best mAP50-95	Best Small mAP50-95
27	27	0.733	0.412	0.266

Table 27: Detection performance with the final configuration.

This was the best-performing training run, achieving an mAP50-95 of 0.412 and exceeding 0.400 for the first time across the experiments. This represents a considerable improvement in overall detection performance compared with the previous configurations. One potential explanation is that combining random horizontal flips and random zoom-ins introduced greater variation into the training images than either augmentation individually. This increased variation may have allowed the model to generalise better to different orientations, positions and scales of fire and smoke objects.

6.8 Hyperparameter Tuning

For the base setting, we use a learning rate of $\eta = 10^{-3}$, a weight decay of 10^{-4}, a backbone learning rate of $0.1 \times \eta$ (Table 2) and augmentation turned on. Each experiment changes one of these per increment. All runs use the same training budget as cross-validation where validation occurs on fold 0 and the runs use the same seed.

Experiment	Best epoch	mAP50	mAP50-95	Smoke / Fire mAP50-95	vs. base
Weight decay 10^{-3}	8	0.356	**0.131**	0.138 / 0.124	+0.012
Base	7	0.334	0.119	0.129 / 0.109	0
Weight decay 10^{-5}	8	0.335	0.115	0.130 / 0.100	−0.004
$\eta = 3 \times 10^{-3}$	7	0.300	0.110	0.126 / 0.094	−0.009
Augmentation off	8	0.305	0.109	0.093 / 0.124	−0.011
$\eta = 3 \times 10^{-4}$	8	0.283	0.103	0.117 / 0.089	−0.016
$\eta = 2 \times 10^{-3}$	8	0.297	0.101	0.095 / 0.108	−0.018
Backbone at $1 \times \eta$	8	0.265	0.093	0.080 / 0.106	−0.026
$\eta = 10^{-4}$	8	0.216	0.071	0.084 / 0.059	−0.048

Table 28: Results of each tuning experiment on validation fold 0.

The learning rate has the biggest effect on mAP50-95 (Table 28). Higher and lower rates both do worse, and 10^{-4} is the worst run, as it may learn too slowly for 8 epochs.

Training the backbone at the full learning rate costs 0.026 mAP50-95, which reinforces using a lower learning rate for the pretrained backbone.

Turning augmentation off causes the model to overfit with a final gap (validation loss – training loss) of 14.1, against 4.8 for the base. Smoke drops from 0.129 to 0.093, while fire slightly improves.

A weight decay of 10^{-3} beat the base by 0.012. This is from one run on one fold, and cross-validation showed the two perform the same (Table 5).

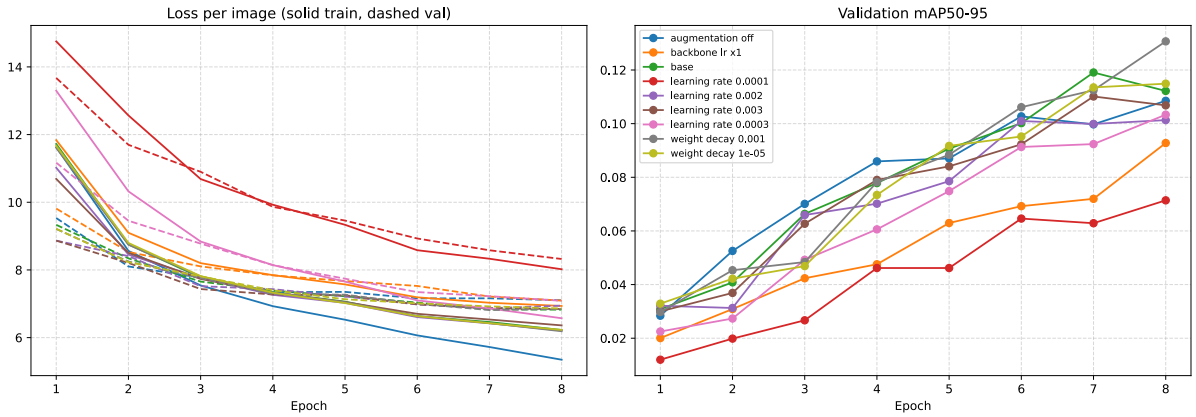


Figure 23: Training (solid) and validation (dashed) loss, and validation mAP50-95, for each tuning experiment.

7 Level of Difficulty

From the difficulty scale provided, we believe our hybrid model lies in the highest diffculty band, with a score of 2 marks. We implemented a hybrid model that combines both ResNet and YOLOv8, building our backbone layer-by-layer, before connecting with the YOLO neck and head using an adapter layer. The neck of the model was also built layer-by-layer, including the top-down FPN and bottom-up PAN passes. To add to this, we also wrote our own training loop with gradient accumulation and a differential learning rate for the backbone. Early-stopping, five-fold cross-validation and nine tuning experiments were also carried out by us while completing the project.

8 Statement of Work

Ciaran Whelan (23370211)

I did the data exploration and analysis of the dataset, examining image characteristics, bounding-box distributions, class distributions and potential biases within the data. I implemented the pretrained ResNet backbone for our hybrid detection model and developed the image preprocessing and augmentation pipeline. I also designed and conducted the independent experiments investigating changes to image resolution, data augmentation and learning-rate scheduling, including plateau-based learning-rate reduction and cosine annealing.
I contributed to the model's training and evaluation pipeline and carried out experiments investigating the effects of different configurations on detection performance. I wrote the corresponding report sections covering data exploration, preprocessing, independent experiments and their results. Finally, I analysed the model's performance on the test set, including detection performance across different object sizes and classes, and compared our results against published research using similar datasets and detection architectures.

Emma O'Reilly (23304472)

I implemented the YOLOv8 neck and head and combined these with the ResNet backbone to create our hybrid model. I implemented the adapter to ensure the outputs of our ResNet backbone match the inputs expected by the YOLO neck. I implemented the neck of the model layer-by-layer, including the top-down FPN and bottom-up PAN passes. I integrated the Ultralytics v8DetectionLoss function into our model and added the mAP50 metric for evaluation. To ensure we could use the imported YOLO loss, I rewrote the dataset to return labels as tensors. I also contributed to the logic for gradient accumulation and added the differential learning rate for the backbone to ensure the hybrid model doesn't lose the benefits of the pretrained ResNet backbone. I added logging per epoch

which saves losses and mAP50 to a CSV file for each training epoch. For the report, I wrote the Network Structure section.

Hersh Singh (23371137)

I created the hyperparameter tuning and cross-validation notebooks. I grouped near-duplicate images with a difference hash for cross-validation, so that no group is split across two folds. I measured how many validation and test images have a near-duplicate in the train split. I ran the nine tuning experiments and both five-fold cross-validation runs on a GPU, and set up my notebooks so that a failed run is retried when a run is resumed (in case of power failure, crashes etc.). I also implemented the final submission notebook, including the test evaluation with and without near-duplicates. In the report, I wrote the Cross-Validation, Model Comparison and Hyperparameter Tuning sections, and the near-duplicate analysis in Results.

9 Generative AI Use

Ciaran Whelan (23370211)

Complete prompt	One-line response summary	How it was used in the submission
How do I expand a dictionary stored in a single CSV cell?	Explained how to convert a dictionary stored as a string into a Python dictionary.	Used to process and extract evaluation metrics stored within CSV files for analysis and visualisation.
Summarise the contributions of these papers to fire detection, highlight architectural differences, and return mAP50-95 results or equivalent metrics for comparison.	Summarised the papers' architectures, reported detection performance	Used to support the literature comparison section and identify architectural differences between our model and existing fire detection models.
Generate BibTeX references for these websites: [...]	Generated BibTeX entries containing publication information and source URLs.	Used to create bibliography entries for documentation, tutorials and external code references consulted during development.
How do I convert pretrained PyTorch weights to my own custom architecture?	Explained how pretrained model weights can be loaded into compatible layers of a custom architecture.	Used to guide the implementation of the pretrained ResNet backbone within our hybrid detection model.
How do I calculate recall similarly to how MeanAveragePrecision is used?	Explained how recall can be extracted from TorchMetrics evaluation results at different IoU thresholds and object sizes.	Used to implement additional recall metrics for analysing detection performance across object sizes and IoU thresholds.
Can you generate me a interactive buttons so click through images so I do not have 18 images displaying vertically in a notebook	Explained how to create interactive image navigation buttons in a Jupyter notebook, allowing images to be displayed individually rather than all at once.	Used to implement an interactive image viewer for examining invalid dataset samples, reducing notebook clutter while allowing users to navigate between images.

Table 29: Ciaran's generative AI prompts, responses and how they were used.

Emma O'Reilly (23304472)

Generative AI tools were used sparsely throughout the project, primarily for explanations, troubleshooting and debugging. The list below outlines all prompts used, along with the responses generated:

Complete prompt	**One-line response summary**	**How it was used in the submission**
"How does a yolo model built layer by layer differ in code to importing the yolo model for use in the hybrid model?"	The response listed the differences, with code examples.	Helped me estimate the difference in difficulty between encorporating an imported model with a predefined backbone and building the model layer by layer - I ultimately decided to build the neck of the model layer-by-layer while importing the YOLO Detect head along with the loss.
"Explain FPN top-down and PAN botom-up in a simple, general context"	The response summarised how the concepts work - with upsampling, downsampling and concatenation, making reference to the research papers that first introduced them.	Helped me understand the logic behind the neck of our hybrid model, and how it can be represented in a diagram.
"What is the SiLU function defined as?"	The exact definition of the function, in mathematical terms and casual language.	Helped me understand why the activation function was appropriate for use in the YOLOv8 model, and how it differs from the ReLU function used in the ResNet backbone.
"How can I add mAP to evaluation metrics?"	Outlined the steps involved with adding the extra metric - the necessary libraries, a type for storing the result, the file the computation should be completed in how to show it in the epoch log.	Used to help implementation of the metric in a structured way, and to ensure the metric was being computed correctly.

Table 30: Generative AI prompts, responses and how they were used.

Hersh Singh (23371137)

I used GenAI for my parts of the project. These are the main prompts I used; a full log of every prompt is available on request.

Complete prompt	One-line response summary	How it was used in the submission
"Integrate everyones code the best we can as a draft for the final Notebook, but its vital if we have the option to use preexisting csvs as results to save computational time"	Built a draft of the final notebook with everyone's code, which loads saved results and only trains what is missing.	As a draft for the final notebook which I then checked and edited.
"Before we do that double check that the outputs ran match the report"	Compared the notebook outputs with the numbers in the report.	Confirm report matches the notebook.
"Lets clean up the markdown for the final notebook draft too, let's remove anything unneccessary and keep it consise"	Shortened the text cells of the notebook.	In the final notebook.
"Is there anything that may appear as vague that I should comment in my segments of the final notebook code? Highlight these as [edit] so I can return to"	Listed lines in my code that needed comments marked [edit].	I wrote the comments for lines labelled [edit].
"Ok, cool, can you update any of my other notebooks with the exact same comments as well then"	Copied my comments into my cross-validation and hyperparameter tuning notebooks.	In the notebooks mentioned.
"Can you check my near-duplicates paragraph against the results and flag discrepencies??"	Checked each number in the paragraph and table against the results and flagged two rounding errors.	Correct the two rounded numbers in my near-duplicates table (Table 12): mAP50 0.860 to 0.859 and mAR100 0.534 to 0.533.

Table 31: Hersh's generative AI prompts, responses and how they were used.

Bibliography

[1] sayedgamal99, "Smoke-Fire-Detection-YOLO." [Online]. Available: https://www.kaggle.com/datasets/sayedgamal99/smoke-fire-detection-yolo

[2] Ultralytics, "Preprocessing Annotated Data." [Online]. Available: https://docs.ultralytics.com/guides/preprocessing-annotated-data/

[3] Ultralytics, "Data Augmentation using Ultralytics YOLO." [Online]. Available: https://docs.ultralytics.com/guides/yolo-data-augmentation/

[4] PyTorch, "ResNet50 — Torchvision Documentation." [Online]. Available: https://docs.pytorch.org/vision/stable/models/generated/torchvision.models.resnet50.html

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep Residual Learning for Image Recognition." [Online]. Available: https://arxiv.org/abs/1512.03385

[6] Ultralytics, "Ultralytics YOLOv8." [Online]. Available: https://docs.ultralytics.com/models/yolov8/

[7] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie, "Feature Pyramid Networks for Object Detection." [Online]. Available: https://arxiv.org/abs/1612.03144

[8] S. Liu, L. Qi, H. Qin, J. Shi, and J. Jia, "Path Aggregation Network for Instance Segmentation." [Online]. Available: https://arxiv.org/abs/1803.01534

[9] PyTorch, "AdamW — PyTorch Documentation." [Online]. Available: https://docs.pytorch.org/docs/stable/generated/torch.optim.AdamW.html

[10] J. Buchner, "ImageHash: A Python Perceptual Image Hashing Module." [Online]. Available: https://github.com/JohannesBuchner/imagehash

[11] scikit-learn, "Cross-validation: Evaluating Estimator Performance." [Online]. Available: https://scikit-learn.org/stable/modules/cross_validation.html#group-k-fold

[12] SciPy, "ttest_rel — SciPy Manual." [Online]. Available: https://docs.scipy.org/doc/scipy/reference/generated/scipy.stats.ttest_rel.html

[13] Lightning AI, "Mean-Average-Precision (mAP) — TorchMetrics Documentation." [Online]. Available: https://lightning.ai/docs/torchmetrics/stable/detection/mean_average_precision.html

[14] W. Ruangsang and P. Pramkeaw, "Cross-Dataset Evaluation of YOLOv8 for Unmanned Aerial Vehicle Fire and Smoke Detection: Benchmark Contamination, Zero-Shot Transfer, and Onboard Deployment on a Low-Cost Airframe," *Drones*, vol. 10, no. 8, p. 635, 2026, doi: 10.3390/drones10080635.

[15] H. Liu *et al.*, "TFNet: Transformer-Based Multi-Scale Feature Fusion Forest Fire Image Detection Network," *Fire*, vol. 8, no. 2, p. 59, 2025, doi: 10.3390/fire8020059.

Code Sources

[16] M. A. Khan, "Implementing YOLO Using ResNet as Feature Extractor." [Online]. Available: https://medium.com/@m.khan/implementing-yolo-using-resnet-as-feature-extractor-5857f9da5014

[17] PyTorch Contributors, "torch.nn.Conv2d – PyTorch 2.14 Documentation." [Online]. Available: https://docs.pytorch.org/docs/2.14/generated/torch.nn.Conv2d.html

[18] PyTorch Contributors, "torch.nn.BatchNorm2d – PyTorch 2.14 Documentation." [Online]. Available: https://docs.pytorch.org/docs/2.14/generated/torch.nn.BatchNorm2d.html

[19] PyTorch Contributors, “torch.nn.MaxPool2d – PyTorch 2.14 Documentation.” [Online]. Available: https://docs.pytorch.org/docs/2.14/generated/torch.nn.MaxPool2d.html

[20] erm, “Random Initialization of Weights with torch.nn.init?.” [Online]. Available: https://discuss.pytorch.org/t/random-initialization-of-weights-with-torch-nn-init/79831

[21] PyTorch Contributors, “ResNet Implementation in Torchvision.” [Online]. Available: https://github.com/pytorch/vision/blob/main/torchvision/models/resnet.py

[22] V. Chugani, “A Gentle Introduction to Learning Rate Schedulers.” [Online]. Available: https://machinelearningmastery.com/a-gentle-introduction-to-learning-rate-schedulers/

[23] C. Whelan, “Out of Distribution Detection in MVTec AD Using DINO Transformers.” [Online]. Available: https://github.com/CiaranW005/UL-Summer-Bursary-2026